

A Novel Covert Timing Channel for Cloud FPGAs

Brian Udugama^{* †}, Darshana Jayasinghe[‡], Hassaan Saadat[§], Aleksandar Ignjatovic[†] and Sri Parameswaran[‡]

^{*}CSIRO Space and Astronomy, PO Box 76, Epping NSW 1710, Australia.

[†]School of Computer Science and Engineering, University of New South Wales, Sydney, Australia.

[‡]School of Electrical and Computer Engineering, University of Sydney, Sydney, Australia.

[§]School of Electrical Engineering and Telecommunications, University of New South Wales, Sydney, Australia.

Abstract—This paper presents a novel covert timing channel (CTC) that enables a malicious entity to exfiltrate data from a benign cloud FPGA user without requiring dedicated outgoing messages from the cloud FPGA, minimizing the detection risk by both the victim and the cloud service provider. The proposed CTC exploits the handshake signals of the Advanced eXtensible Interface (AXI) protocol and inter-packet delay of the Internet to establish the CTC from a cloud Field-Programmable Gate Array (FPGA) to an off-cloud computer. This paper analyzes the bit-error rate (BER) of the AXI-based CTC under varying conditions and demonstrates its effectiveness in truly enabling remote power analysis attacks on cloud services, such as Amazon Web Services Elastic Compute Cloud (AWS EC2). The proposed CTC achieves a BER as low as 0.01988%.

Index Terms—Covert Timing Channels, Cloud FPGA, AXI Protocol, Remote Power Analysis Attacks.

I. INTRODUCTION

Field-Programmable Gate Arrays (FPGAs) are increasingly deployed in cloud environments, vitalizing the importance of investigating remotely deployable hardware-level security threats. Cloud FPGA services, such as Amazon Web Services Elastic Compute Cloud (AWS EC2) F1 instances, provide users with FPGA resources in a virtual environment, restricting direct user interaction with peripherals for consistency and security. In AWS EC2 F1 instances, the Advanced eXtensible Interface (AXI) protocol [1] manages communication between the AWS-provided shell [2], which controls peripherals, and the user-generated custom logic circuit.

This paper introduces, for the first time, a covert timing channel (CTC) [3] that enables data exfiltration from a cloud FPGA user, without requiring dedicated outgoing messages from the cloud FPGA. The proposed CTC is composed of two phases. In the initial phase, the CTC leverages the handshake signals of the AXI protocol to covertly transfer data from a cloud FPGA to an off-FPGA resource. In the second phase, it exploits inter-packet delays over the Internet to continue the covert transfer from the cloud resource to an off-cloud computer, creating a CTC from the cloud FPGA to the off-cloud computer. To demonstrate the threat posed by the proposed CTC, power consumption data gathered from a cryptographic circuit deployed on a cloud FPGA are exfiltrated to an off-cloud computer, where it is analyzed to extract the secret key of the cryptographic circuit. Moreover, this paper evaluates the efficacy of the CTC by analyzing the bit-error rate (BER) of the proposed CTC under varying design and operational conditions. Hence, the CTC presented in this paper poses a significant security risk to benign cloud FPGA users, exhorting users to take preventive measures.

Paper Contributions:

- Presents the first covert communication channel to exfiltrate data from a cloud FPGA without requiring explicit outgoing messages from the FPGA.
- Demonstrates the possibility of exploiting the handshake signals of the AXI protocol to establish a CTC on an FPGA.

- Signifies the importance of CTC to fully realize a remote power analysis (RPA) attack [4], emphasizing the implications for cloud FPGAs and their users.
- With substantial experimentation, analyzes the effectiveness of the CTC with respect to BER under design and operating condition changes.

Paper Organization: The rest of this paper is organized as follows. Section II provides key background knowledge for understanding this paper, while Section III discusses related works from the literature. The assumed threat model is detailed in Section IV. Section V presents a performance and effectiveness evaluation of the proposed CTC while the subsequent Section VI presents the design of signal processing techniques associated with the CTC. Section VII concludes this paper.

II. BACKGROUND

Architecture of AWS EC2 F1 Instances: AWS EC2 F1 instances allow the deployment of FPGA-based hardware accelerators on cloud FPGAs. Each AWS EC2 F1 instance includes virtual central processing units (vCPUs) running a general-purpose operating system and FPGA cards implementing user-defined custom hardware accelerators. Configuring an FPGA on an AWS EC2 F1 instance requires loading an Amazon FPGA Image (AFI) onto the FPGA. First, the users are required to compile the custom logic circuit to an intermediate design checkpoint (DCP), and then submit the DCP to AWS to perform design rule checks (DRCs). A DCP that passes all the DRCs, conducted by AWS, is transformed into an AFI by wrapping the custom logic circuit of the DCP with an AWS-designed shell [2]. The shell implements the interfaces for FPGA external peripherals, PCIe, DRAM, and interrupts [2]. User interactions with PCIe and DRAM are to be conducted through the shell interfaces, which are implemented using the AXI protocol [2].

AXI Protocol: AXI protocol is an on-chip interconnect specification [1] that utilizes five unidirectional channels, namely Write Address (AW), Write Data (W), Write Response (B), Read Address (AR) and Read Data (R). Each AXI channel uses a pair of handshake signals to indicate the readiness of the endpoints for a data exchange. The source asserts the VALID signal when it has loaded valid data onto the AXI connection. The READY signal is asserted by the destination when the destination is ready to accept incoming data. In this manner, the handshake allows both endpoints to control the rate of data exchange on an AXI channel and thereby control the rate of data exchange on the AXI connection.

In the AXI protocol, a transfer refers to a sequence of signals that perform a data exchange with a single handshake. A transaction refers to a sequence of transfers that begins with an address transfer, followed by one or more data transfers. In a write transaction, a response transfer follows all the data transfers.

Correlation Power Analysis (CPA): CPA is a statistical method to analyze power measurements to extract the secret key of a cryptographic module [5]. In CPA, the attacker partitions the secret

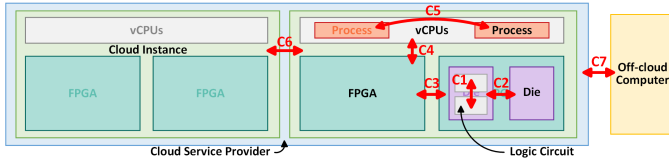


Fig. 1. Covert Communication Channels in Cloud Context

key into bytes, models the power consumption for each byte guess and calculates correlation between observed and modeled power consumption for each one of the byte guesses [6]. With sufficient observations, the correlation factor of the correct guess stands out, revealing the secret key.

Low-Density Parity-Check (LDPC) Codes: LDPC is a scheme of error-correcting codes designed to recover errors caused by transmission over noisy channels [7]. An LDPC code is defined by a parity-check matrix, typically generated randomly under certain constraints. When decoding using an LDPC code, the parity-check matrix is used to iteratively refine an initial guess of the error-correct output for a predefined number of times.

III. RELATED WORK

The schematic in Fig. 1 depicts various covert communication channels implementable in a cloud service hosting FPGAs. The bidirectional arrow labeled C1, in Fig. 1, represents a covert communication channel between two disjoint logic circuits on an FPGA die within a cloud FPGA. A covert communication channel between two dies, also known as Super Logic Regions (SLRs), within an FPGA is represented by the bidirectional arrow labeled C2, in Fig. 1. Similarly, the bidirectional arrows labeled C3, C5 and C6, in Fig. 1, correspondingly illustrate covert communication channels between two FPGAs within a cloud instance, two vCPUs within a cloud instance and two cloud instances within a cloud service.

The covert communication channels represented by C4 and C7, in Fig. 1, are fundamentally different from the other covert communication channels due to the asymmetry of the endpoints. The upward C4 covert communication channel, as illustrated in Fig. 1, requires a transmitter on the FPGA and a receiver on vCPUs, while the downward covert communication channel requires the reversed endpoint configuration. Thus, a covert communication channel that is designed to infiltrate data to a cloud FPGA fails to exfiltrate data from a cloud FPGA. In C7, the leftward covert communication channel has the transmitter implemented on an off-cloud computer and the receiver implemented within the cloud service provider. Conversely, the rightward C7 covert communication channel features a transmitter within the cloud service provider and a receiver on an off-cloud computer.

Giechaskiel et al. [8] demonstrated a covert communication channel exploiting crosstalk between adjacent long wires in an FPGA. A malicious entity capable of routing a long wire next to a lengthy piece of wire belonging to an unsuspecting victim residing on an FPGA could exfiltrate data from the victim by using crosstalk-induced signal propagation delay variations [8]. This methodology [8] enables covert communication between two logically isolated circuits deployed on an FPGA. Gnad et al. [9] presented a voltage-based covert communication channel that exploits the power delivery network (PDN) of the FPGA. In this method [9], the malicious user deploys power wasters to modulate data into voltage fluctuations that occur on an FPGA and capture the voltage fluctuations using a logically isolated circuit. Both of the above methods [8], [9] illustrate implementations of the C1-labeled covert communication channel in Fig. 1.

Using voltage fluctuations that traverse the PDN of an FPGA, Giechaskiel et al. [10] established a covert communication channel between the SLRs of an FPGA. The authors [10] demonstrated the deployment of the covert communication channel in Amazon and Huawei cloud computing services, achieving a minimum bandwidth of 4.6 Mbps and an accuracy over 97.6%. Covert communication channels with similar capabilities are represented by C2 in Fig. 1.

Giechaskiel et al. [11] further demonstrated a methodology to exploit a shared power supply unit to construct a covert communication channel capable of infiltrating data into a cloud FPGA from a central processing unit (CPU), a graphics processing unit (GPU) or another FPGA. The authors [11] modulated the data as voltage fluctuations induced by power-wasting techniques, and demonstrated that induced voltage fluctuations are observable from an FPGA sharing the power supply unit with the component inducing voltage fluctuations. As per Fig. 1, the FPGA to FPGA covert communication channel [11] fits category C3, the transmitter being a power waster circuit constructed on an FPGA and the receiver being a voltage sensing circuit constructed on another FPGA. The CPU to FPGA covert communication channel [11] implements the C4 covert communication channel in the downward direction, as shown in Fig. 1, where CPUs adjust the load, thereby the instantaneous power, to transmit information while voltage sensing circuits on FPGAs measure the voltage fluctuations to receive covertly transmitted information.

Over several decades, using response time as a covert communication channel has been investigated by researchers. In 1987, Girling [12] demonstrated the use of inter-packet delay of a local area network (LAN) to establish a covert communication channel; a concept later applied in software-defined networks (SDNs) [13] and the Internet [14]. Lee et al. [14] demonstrated a covert communication channel spanning several thousand miles over the Internet, achieving 81 kbps bandwidth and less than 10% BER using a Software-defined Network Interface Card (SoNIC) [15]. In FPGA-equipped cloud instances, a SoNIC can be constructed within the FPGA, which can be exploited as proposed by Lee et al. [14] to implement C7 in Fig. 1. Ristenpart et al. [16] and Pessl et al. [17] have proposed techniques that implement the covert communication channel labeled C6, in Fig. 1. Ristenpart et al. [16] exploited the behavior of caches, while Pessl et al. [17] exploited the addressing mechanism used in Dynamic Random Access Memory (DRAM) to exfiltrate data from one cloud instance to another cloud instance that reside on the same physical computer. Alagappan et al. [18] developed a covert communication channel between CPU processes, represented by C5 in Fig. 1, utilizing power governors of a CPU to covertly modulate data into the CPU operating frequency, achieving up to 20 bps.

In the literature, covert communication channels that implement the upward, as illustrated in Fig. 1, covert communication channel C4 (FPGA to vCPU) are scarce and often tied to specific attack scenarios. Purnal et al. [19] exploited the behavior of CPU caches to construct an FPGA to the CPU covert communication channel. This method [19] uses a malicious process, running on the CPU, to prime the caches by loading specific symbols, which the FPGA then selectively overwrites based on the information to be exfiltrated. Finally, the malicious process measures the read times, revealing the symbols overwritten by the FPGA. The authors [19] achieved a 264 kbps bandwidth with a 1.43% BER and also claimed that their proposition is less detectable to runtime analysis as the FPGA design belonging to the malicious user can rapidly overwrite symbols written by the malicious process, without affecting the runtime of a process.

Despite the covert communication channel by Purnal et al. [19]

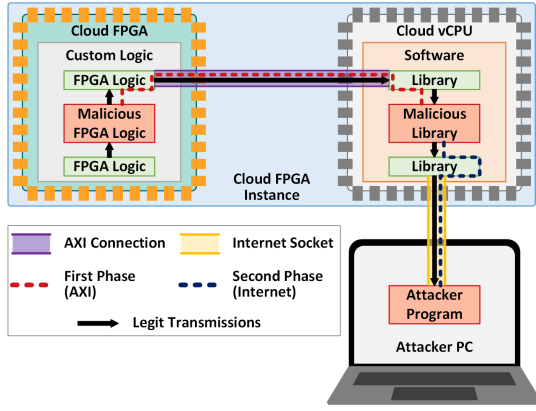


Fig. 2. Threat Model

is capable of staying undercover within the software domain, it lacks effective concealment on the hardware. The authors [19] in their threat model, assume a malicious user controls at least one CPU core and a secondary device such as an FPGA, dispatching operations to both software and hardware while sharing memory between them. In cloud FPGA instances, where AXI interfaces are limited, overt transmission over AXI poses the risk of exposing the covert communication channel to the cloud service provider or other users of the cloud FPGA instance. The covert communication channel from FPGA to CPU proposed by Weissman et al. [20] also relies on cache exploitation and data transfers between the FPGA and the memory, making the covert communication channel similarly vulnerable to detection.

Establishing effective FPGA to vCPU and cloud to off-cloud covert communication channels is pivotal for malicious entities residing on cloud FPGAs, to exfiltrate data from the cloud FPGA to an off-cloud resource. The existing literature lacks effective covert communication channels for establishing the FPGA to vCPU connection in the context of cloud FPGAs, with existing techniques [19], [20] exhibiting notably specific attack scenarios. Furthermore, the literature lacks effectiveness assessment of covert timing channels (CTCs) over the Internet, in the context of cloud. This paper, for the first time, proposes a CTC effective for exfiltrating data from a cloud FPGA to an off-cloud computer. In the first phase, the proposed CTC exploits the VALID and READY handshake signals of the AXI protocol to covertly transmit data from the FPGA to the vCPUs of a cloud FPGA instance. In the second phase, the proposed CTC exploits the inter-packet delay over the Internet to covertly transmit data from the cloud to an off-cloud computer.

IV. THREAT MODEL

The threat model in this paper focuses on a cloud FPGA instance, such as an AWS EC2 F1 instance, targeted by an attacker residing external to the cloud service. As depicted in Fig. 2, the cloud FPGA instance hosts a system with a malicious hardware-software suite injected by the attacker. The malicious hardware-software suite consists of a logic circuit (labeled Malicious FPGA Logic) integrated into the custom logic of an unsuspecting victim, and a software library (labeled Malicious Library) invoked by the software process executing on cloud vCPUs. The malicious hardware-software suite aims to covertly transfer data from the cloud FPGA to the cloud vCPUs and thereafter to the off-cloud personal computer (PC) controlled by the attacker, by implementing the upward C4 covert communication channel of Fig. 1 and the rightward C7 covert communication channel of Fig. 1, respectively.

Complex FPGA logic circuits often use third-party intellectual property (IP) cores like encryption engines, audio/visual pipelines and arithmetic units [21]. With cloud FPGA instances offering powerful vCPUs, users can offload vCPU-efficient tasks onto vCPUs, paving the way for third-party hardware-software IPs to become prevalent in the near future. To infiltrate the infrastructure of an unsuspecting victim, the malicious hardware-software suite disguises itself as a useful hardware-software IP by providing services verified through test vectors. The functionality of the Malicious FPGA Logic and the Malicious Library is verified individually by supplying test vectors and comparing the outputs with the expected algorithmic output. The Malicious FPGA Logic does not necessarily have a direct connection to the Malicious Library, blocking the opportunity for output manipulation. However, the Malicious FPGA Logic can alter the time it takes to validate the data transferred between the FPGA and vCPU sides in the cloud FPGA instance.

In comparison to existing work, this paper assumes a holistic and realistic attack scenario with broader applicability to cloud FPGAs. An attacker with access to a small logic area on the cloud FPGA and a minimal portion of runtime on vCPUs can deploy both phases of the proposed CTC, without requiring the ability to share memory between software and hardware. Prior works [19], [20] typically assumed that the malicious user could dispatch operations to both software and hardware while sharing memory between them. *Furthermore, note that the attacker does not need to be an endpoint of Internet communication to intercept inter-packet delay.* It is sufficient for the attacker to merely eavesdrop on an ongoing Internet traffic.

V. PERFORMANCE EVALUATION

A. Experiment Setup

The experiment setup implements the threat model in an AWS EC2 F1 instance to evaluate the performance of the proposed CTC. The AWS EC2 F1 instance featured eight vCPUs derived from an Intel Xeon E5-2686 v4 multiprocessor and an AMD Xilinx UltraScale+ XCVU9P-FLGB2104-2-I FPGA. To mimic FPGA logic of the victim, a simple echo server was implemented on the FPGA and the AXI Connection, as shown in Fig. 2, was used to receive and echo back messages between the FPGA and the vCPU. In addition, the transmitter of the first phase of the CTC was placed at the FPGA side interface of the AXI Connection (to be specific, R channel of the AXI Connection). The FPGA logic circuit was synthesized and implemented using Xilinx Vivado v2021.2, available through the FPGA Developer AWS Machine Image (AMI) at the time of the experiment. *Please note that the FPGA logic was clocked at 125 MHz.*

The vCPU side of the AWS EC2 F1 instance implemented a software service to route User Datagram Protocol (UDP) network clients to the echo server, mimicking the software belonging to the victim. Moreover, the receiver for the first phase and the transmitter for the second phase of the CTC too were implemented on the vCPUs. The receiver of the first phase of the CTC utilized the RDTSCP instruction (Read Time Stamp Counter and Processor ID) [22] to capture precise time stamps from the Time Stamp Counter (TSC) of the processor [23]. Modern Intel 64 and IA-32 processors, including vCPUs in AWS EC2 F1 instances, feature an advanced Invariant TSC [23] which continues to increment regardless of the state of the processor (sleep or active) and remains invariant to the core executing the RDTSCP instruction [23]. Inspired by [12], the transmitter for the second phase of the CTC performed the covert transmission by selectively delaying the transmission of UDP packets echoed back from the server. The attacker PC analyzed the inter-packet delay to recover the covertly transmitted information.

TABLE I
COVERT TIMING CHANNEL PERFORMANCE – RPA ATTACK EFFICACY

Power Trace Set	Bit Errors (out of 204.8M bits)	Power Traces to Recover Key	Increase in Power Traces to Recover Key (vs. Reference)
Reference	Not Applicable	17,100	Not Applicable
After CTC first phase	46,592	18,600	9%
After CTC second phase	1,302,835	21,300	24%

To demonstrate the effectiveness of the proposed CTC in RPA attacks, the exact set of power consumption traces from the RPA attack specific Power to Pulse Width Modulation Sensor (PPWM), published as artifacts [5], was used. PPWM is an area-efficient on-chip sensor that produces a single bit output per measurement of power consumption [5]. This characteristic enables attackers to effectively utilize a lower-bandwidth channel, such as the proposed CTC, to discreetly transfer power consumption data from a cryptographic engine deployed on a cloud FPGA to an off-cloud PC, where key inference can be performed using Correlation Power Analysis (CPA). *Please note the power consumption traces correspond to an unprotected 128-bit Advanced Encryption Standard (AES) circuit.*

In this experiment, the set of power consumption traces was transmitted via the first phase (AXI) and the second phase (Internet) of the CTC. After each phase, the output was collected at the receiver end and subjected to CPA for key inference. A CPA was also conducted on the original set of power consumption traces as reference.

In the first phase of the CTC, a logic 1 was covertly transmitted by delaying the VALID signal of the AXI Connection by 2 clock cycles, while a logic 0 was transmitted without inducing a delay. In the second phase of the CTC, a logic 1 was transmitted by delaying a UDP packet by 2 ms, with no added delay for the transmission of a logic 0. Synchronization between transmitter and receiver is essential for accurate demodulation [24]. However, as UDP is a best-effort service without packet delivery guarantees [25], packet loss can disrupt synchronization, hindering the demodulation of subsequent transmissions. To address the aforesaid issue, the transmitter introduced a 100 ms delay after every 2,000th unit of data, defined as a *block*. A *block* contains data bits along with a set of parity bits and reference bits to aid in demodulating and decoding the data bits. The receiver advanced the computation to the start of a *block* upon observing the increased delay. Missing observations were padded with zeros. Data in 2,000 *blocks* equated to 100 power consumption traces from PPWM and the frequency of inserting the 100 ms delay was experimentally determined. In this paper, a streaming protocol, feeding continuous data, is assumed above the *block* level of the protocol stack. However, the streaming protocol can be replaced by a packet-based protocol [26], [27], if required.

B. Results

Table I summarizes the key experiment result of this paper. The first column of Table I identifies the power consumption trace sets used for CPA. The next column presents the number of bit errors observed in the power consumption traces, which in total spanned 204.8 million bits. The subsequent column displays the number of power consumption traces needed to successfully recover the secret key of the targeted cryptographic engine. The last column quantifies, as a percentage, the increase in the number of power consumption

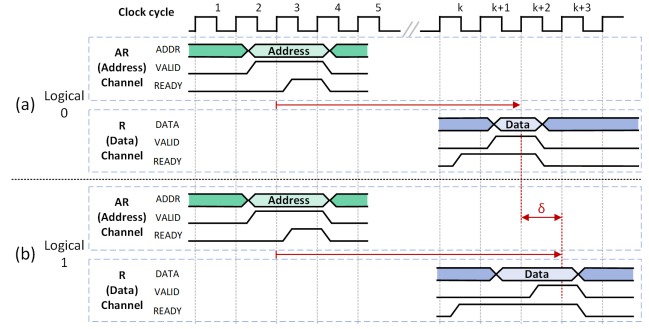


Fig. 3. AXI Read Transaction - Timing Diagrams

traces required for complete key recovery relative to the reference trace set.

In this experiment, the reference trace set needed 17,100 power consumption traces to fully recover the cryptographic key. After transmission through the first phase of the CTC, this number increased by 9% to 18,600. Subsequently, the final output after the second phase of the CTC required 21,300 power consumption traces, marking a 24% increase compared to the reference. Overall, this experiment demonstrated the possibility of using the proposed CTC together with a PPWM sensor to perform an RPA attack that does not require physical access to measure or retrieve power consumption traces from a cloud FPGA to an off-cloud PC. *Please note that BERs of first phase and second phase of the CTC were 0.02275% and 0.6134%, respectively.*

VI. THE PROPOSED COVERT TIMING CHANNEL – DESIGN PARAMETERS

A. Overview of Design Parameters

This section evaluates the impact of key design parameters on the first phase of the CTC. The authors expect similar effects to be observed in the second phase of the CTC. The transmitter of the first phase of the CTC is implemented as a logic circuit on the FPGA that can manipulate the timing of the VALID signal of the AXI Connection and the receiver of the CTC is a software component integrated within the compute pipeline on vCPUs, which gauges the time consumed by a read transaction. The transmission medium of the CTC is the time consumption of transactions performed through the AXI Connection.

When read transactions occur through the AXI Connection, the transmitter of the CTC bit-by-bit injects the *block* into the transmission medium, the time consumption of read transactions, by delaying the VALID signal of the R channel by an arbitrary number of clock cycles. In the bit modulation scheme for the CTC, a logic 0 is transmitted by inducing no delay in the VALID signal. Conversely, a logic 1 is transmitted by delaying the VALID signal by δ clock cycles. Upon completing the read transaction, the receiver residing on the vCPU side measures the elapsed time and uses the elapsed time measurements of the modulated reference bits to demodulate the data bits and the parity bits. Afterwards, the covertly transmitted data bits are decoded using the intrinsic information from the received data bits and the extrinsic information from the received parity bits. *Note that the number of delay clock cycles, δ , is chosen arbitrarily by the attacker at the design time of the CTC.*

The timing diagrams (a) and (b) in Fig. 3 correspondingly show the behavior of the signals on the AR and R channels of the AXI Connection for covertly transmitting a logic 0 and a logic 1. The timing diagrams in Fig. 3 assume $\delta = 1$ and a read transaction

Algorithm 1 Algorithm for Decoding a Block

```

1: function DECODE( $[D_1, D_2, D_3, \dots, D_n]$ )
2:    $m \leftarrow$  row count of  $H$ 
3:    $E \leftarrow [D_1, D_2, D_3, \dots, D_n]$ 
4:    $L \leftarrow \begin{bmatrix} 0 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 \end{bmatrix}_{m \times n}$ 
5:   for  $i \leftarrow 1, \dots, n$  do
6:     for  $j \leftarrow 1, \dots, m$  do
7:       if  $H[j, i] = 1$  then
8:          $L[j, i] \leftarrow D_i$ 
9:   for  $\text{iterations} \leftarrow 1, \dots, 8$  do
10:    for  $j \leftarrow 1, \dots, m$  do
11:       $\text{row} \leftarrow [L[j, 1] \ L[j, 2] \ \dots \ L[j, n]]$ 
12:       $\text{pos} \leftarrow 0, \ \text{min}_1 \leftarrow \infty$ 
13:      for  $k \leftarrow 1, \dots, n$  do
14:        if  $(\text{min}_1 > |\text{row}[k]| \wedge H[j, k] = 1)$  then
15:           $\text{min}_1 \leftarrow |\text{row}[k]|$ 
16:           $\text{pos} \leftarrow k$ 
17:       $\text{min}_2 \leftarrow \infty$ 
18:      for  $k \leftarrow 1, \dots, n$  do
19:        if  $(\text{min}_2 > |\text{row}[k]| \wedge H[j, k] = 1 \wedge \text{pos} \neq k)$  then
20:           $\text{min}_2 \leftarrow |\text{row}[k]|$ 
21:       $\text{sign} \leftarrow 1$ 
22:      for  $k \leftarrow 1, \dots, n$  do
23:        if  $(H[j, k] = 1)$  then
24:           $\text{sign} \leftarrow \text{sign} \times \text{sgn}(\text{row}[k])$ 
25:       $\text{row}_r \leftarrow [0 \ 0 \ \dots \ 0]_{1 \times n}$ 
26:      for  $k \leftarrow 1, \dots, n$  do
27:        if  $(H[j, k] = 1)$  then
28:          if  $(k = \text{pos})$  then
29:             $\text{row}_r[k] \leftarrow \text{min}_2 \times (\text{sign} \times \text{sgn}(\text{row}[k]))$ 
30:          else
31:             $\text{row}_r[k] \leftarrow \text{min}_1 \times (\text{sign} \times \text{sgn}(\text{row}[k]))$ 
32:       $[L[j, 1] \ L[j, 2] \ \dots \ L[j, n]] \leftarrow \text{row}_r$ 
33:       $E \leftarrow E + [\sum_{i=1}^m L[1, i] \ \sum_{i=1}^m L[2, i] \ \dots \ \sum_{i=1}^m L[n, i]]$ 
34:       $L \leftarrow \begin{bmatrix} E[1] & E[2] & \dots & E[n] \\ \vdots & \vdots & \ddots & \vdots \\ E[1] & E[2] & \dots & E[n] \end{bmatrix}_{m \times n} - L$ 
35:   return  $E$ 

```

consumes k clock cycles. It should be emphasized that the value of k can vary across read transactions.

The receiver of the CTC serves two main functions: (1), demodulating the received signal and (2), decoding the received information. In the demodulation step, the receiver measures the time taken to perform each of the read transactions over the AXI Connection and remaps the expected value of a transmission of logic 1 and the expected value of a transmission of logic 0 to the real numbers -1 and 1 , respectively. The remapping is chosen to simplify the subsequent decoding step and is influenced by binary phase-shift keying (BPSK) modulation [28]. In BPSK, a digital signal is modulated into an analog carrier signal by shifting the phase into one of the two (hence, binary) phase offsets.

1) *Decoding at the Receiver:* The algorithm shown in Algorithm 1 is used for decoding a *block* and it is derived from the min-sum algorithm used in the LDPC code schemes [29]. Min-sum algorithm produces a minimal set of simple operations for decoding a BPSK transmission over an additive white Gaussian noise (AWGN) channel and the inherent resource efficiency of the algorithm is essential to preserve the stealthiness of the CTC. The parity check matrix used in experiments is defined in Equation 1 and is referred to as H , in Algorithm 1.

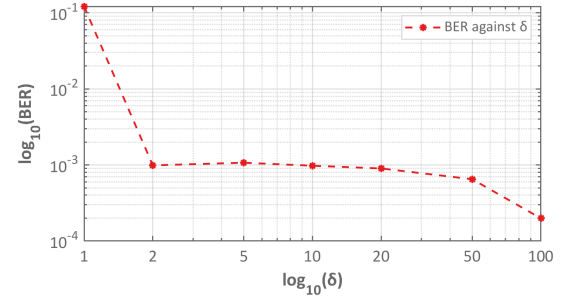


Fig. 4. Variation in Bit-Error Rate – δ

Algorithm 1 is structured as a function named DECODE, which takes an array, $[D_1, D_2, D_3, \dots, D_n]$, as input. At the start of the DECODE function, m , E and L are initialized to the number of rows in the parity check matrix, the input array and an $m \times n$ matrix of zeros, respectively. Then, each element $L[j, i]$ of matrix L is set to D_i , the input array element at the index corresponding to the column index of $L[j, i]$, if $H[i, j]$ equals 1 (lines 5 - 8). The subsequent section of the DECODE function iteratively improves the beliefs stored in array E , aiming to recover the originally transmitted bits. Based on preliminary experiments, the iterative process (lines 10 - 34) required eight rounds for the optimal result.

$$H = \begin{bmatrix} 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \quad (1)$$

The iterative process transforms the matrix L by replacing each of its rows with a replacement row (row_r) computed using the values of the row being replaced. Firstly, the algorithm finds the minimal value (min_1) among the elements of row that have a 1 stored in the identical position of H (lines 13 - 16). Simultaneously, pos is updated with the position of min_1 (line 16). Next, a second minimal value (min_2) is found among the elements of row , excluding the element in position pos , that have a 1 stored in the identical position of H (lines 18 - 20). Then, the product of the signs of all elements of row that have 1 stored in the identical position of H is computed (lines 22 - 24) and stored in sign . The last step in constructing row_r , populates an array of zeros of length n with $\text{min}_2 \times (\text{sign} \times \text{sgn}(\text{row}[k]))$ if $k = \text{pos}$ and $\text{min}_1 \times (\text{sign} \times \text{sgn}(\text{row}[k]))$ otherwise, provided that the identical position of H is 1. After constructing row_r , row of L is replaced by row_r and the computation moves to the next row until all rows are transformed. Then, the array E is updated by adding the column sum of L to E (line 33). As the final step of the iterative process, L is updated by subtracting L from a matrix that is generated by replicating E , m number of times to match the height of L . The final output of Algorithm 1 is the values stored in E after the eighth round.

After executing Algorithm 1, the error-corrected data bits received can be inferred by observing the sign of the corresponding elements of E . If the sign of the element is negative, the bit is inferred as a logic 1 and if the sign of the element is positive or zero, the bit is inferred as a logic 0. In this manner, bits that are covertly transmitted by the transmitter are inferred at the receiver, establishing

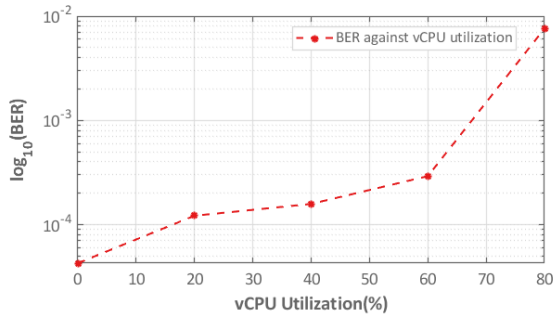


Fig. 5. Variation in Bit-Error Rate – vCPU Utilization

a CTC between the FPGA side and the vCPU side of a cloud FPGA instance. The experiments in this paper were based on $n = 16$ and *blocks* containing 2 data bits.

B. Impact of δ on Bit-Error Rate

To evaluate the dependence of the BER of the CTC on δ , the number of delay clock cycles induced on the VALID signal to covertly transmit a logic 1, multiple experiments were carried out by varying δ between 1 and 100. The result of this experiment is shown in Fig. 4, with δ on the x-axis and BER on the y-axis, both presented on the base-10 logarithmic scale ($\log_{10}$). The graph in Fig. 4 shows that the CTC is effective for all δ values tested in the experiment. Notably, the BER decreases sharply when δ increases from 1 to 2 and an overall downward trend continues, reaching a minimum BER of 0.0198% at $\delta = 100$. The decrease in BER, as δ increases, can be attributed to improved distinguishability between covertly transmitted logic 1s from covertly transmitted logic 0s and the sharp decline in BER from $\delta = 1$ to $\delta = 2$ can be hypothesized as a consequence of exceeding the ambient noise level of the AWS EC2 F1 instance.

In this experiment, changing the value of δ impacted the time taken to complete a read transaction. However, the interval between consecutive read transactions depended on the frequency of echo requests from the client side, regulated by the scheduling of processes on the vCPUs and therefore unaffected by the value of δ . It is important to highlight that the bandwidth of the CTC is influenced by both the intra-read and inter-read transaction delays. When changing δ from 1 to 100, no discernible pattern was observed in the overall time required to execute each experiment, indicating the prominence of the uncertainty associated with the inter-read transaction delay.

C. Impact of vCPU Utilization on Bit-Error Rate

This experiment examined the impact of the vCPU utilization on BER by placing an artificial CPU-intensive workload on the vCPU side of the cloud FPGA instance. Using the `taskset` command, the workload was assigned to the same vCPU as the receiver of the CTC, while CPU utilization of the workload was managed with the `cpulimit` utility. δ was set to 10 for this experiment.

As depicted in Fig. 5, with vCPU utilization on the x-axis and BER (in the $\log_{10}$ scale) on the y-axis, BER shows an upward trend with increasing vCPU utilization. This observation aligns with expectation, as the receiver of the CTC is unable to precisely capture the timestamp when the artificial workload is scheduled on the vCPU. Consequently, inaccuracies in the recorded time intervals lead to decoding errors and demodulation errors, causing higher BER. vCPU utilization becomes relevant to the BER only when the artificial workload interrupts an ongoing read transaction. *Note that, scheduling the artificial workload between two read transactions does not lead to bit errors and as seen from Subsection VI-B, the inter-read transaction*

TABLE II
VARIATION IN BIT-ERROR RATE – MODULATION TECHNIQUE

δ	BER		Relative Improvement as Percentage
	Proposed Scheme (LDPC-based)	Repetition Code	
1	12.140%	22.510%	185%
2	0.099%	0.144%	146%
5	0.048%	0.066%	138%
10	0.098%	0.116%	119%
20	0.090%	0.191%	213%
50	0.065%	0.158%	243%
100	0.020%	0.147%	739%

delay was prominent, reducing the likelihood of the artificial workload interrupting an ongoing read transaction. Therefore, in contrast to approaches that rely on modulating covert information via inter-read transaction delay, the proposed technique, which encodes covert information by modulating the duration required to complete a read transaction, offers enhanced reliability for the attacker.

D. Impact of Modulation Technique on Bit-Error Rate

In order to assess the effectiveness of the LDPC-based error correction scheme used in the decoding algorithm, an experiment was carried out by augmenting Algorithm 1 to implement a decoder based on repetition code. The structure of a *block* was also changed, to facilitate the repetition code and the error corrected two data bits of the *block*, d_1 and d_2 , were calculated according to Equation 2 and Equation 3, respectively. The experiment was repeated for different values of δ in the range of 1 to 100.

$$d_1 = \begin{cases} 0, & \text{if } \frac{(d_{11}+d_{12}+d_{13}+d_{14}+d_{15}+d_{16}+d_{17}+d_{18})}{8} > 0 \\ 1, & \text{otherwise} \end{cases} \quad (2)$$

$$d_2 = \begin{cases} 0, & \text{if } \frac{(d_{21}+d_{22}+d_{23}+d_{24}+d_{25}+d_{26}+d_{27}+d_{28})}{8} > 0 \\ 1, & \text{otherwise} \end{cases} \quad (3)$$

Table II presents the results of the experiments evaluating LDPC-based and repetition code schemes for the CTC. The first column of Table II represents the values of δ used in the experiment. The second and third columns show the BER obtained from the experiment using the proposed LDPC-based scheme and repetition code, respectively. The fourth column shows the improvement in BER as a percentage, calculated by dividing the repetition code BER by the LDPC-based BER. The results in this section show that the proposed LDPC-based scheme outperformed the repetition code, even though it is composed of a minimal set of simple operations. In the pair of experiments where $\delta = 100$, the BER improvement of the proposed LDPC-based scheme is over seven folds compared to the repetition code. Hence, these results portray the ability of the proposed LDPC-based scheme to effectively decode *blocks* while maintaining the stealthiness of the CTC.

VII. CONCLUSION

This paper introduced, for the first time, a CTC that provides an attacker with the capability to covertly transfer data from a cloud FPGA to an off-FPGA resource. Using experiments, this paper demonstrated the application of the CTC in exfiltrating power consumption traces from an on-chip sensor on a cloud FPGA to an off-cloud attacker PC for performing RPA attack. Further, an analysis on the BER of the proposed CTC under various operating conditions was also presented. The findings emphasize the need for increased vigilance and security measures from both cloud FPGA providers and users to protect logic circuit designs deployed on cloud FGAs.

REFERENCES

- [1] Arm Limited, “Learn the architecture - an introduction to AMBA AXI,” <https://developer.arm.com/documentation/102202/0300/AXI-protocol-overview>, 2023.
- [2] D. Patel, “AWS shell interface specification,” https://github.com/aws/aws-fpga/blob/master/hdk/docs/AWS_Shell_Interface_Specification.md, 2023.
- [3] R. W. Shirey, “Internet Security Glossary, Version 2,” <https://www.rfc-editor.org/info/rfc4949>, Aug. 2007.
- [4] F. Schellenberg, D. R. Gnad, A. Moradi, and M. B. Tahoori, “Remote inter-chip power analysis side-channel attacks at board-level,” in *2018 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2018, pp. 1–7.
- [5] B. Udugama, D. Jayasinghe, H. Saadat, A. Ignjatovic, and S. Parameswaran, “A power to pulse width modulation sensor for remote power analysis attacks,” *TCHES*, vol. 2022, no. 4, p. 589–613, Aug. 2022, artifact available at <https://artifacts.iacr.org/tches/2022/a17>.
- [6] E. Brier, C. Clavier, and F. Olivier, “Correlation power analysis with a leakage model,” in *CHES 2004*, 2004, pp. 16–29.
- [7] R. Gallager, “Low-density parity-check codes,” *IRE Transactions on Information Theory*, vol. 8, no. 1, pp. 21–28, 1962.
- [8] I. Giechaskiel, K. B. Rasmussen, and K. Eguro, “Leaky wires: Information leakage and covert communication between FPGA long wires,” in *13th ASIACCS*, 2018, p. 15–27.
- [9] D. R. E. Gnad, C. D. K. Nguyen, S. H. Gillani, and M. B. Tahoori, “Voltage-based covert channels using FPGAs,” *ACM Trans. Des. Autom. Electron. Syst.*, vol. 26, no. 6, jun 2021.
- [10] I. Giechaskiel, K. Rasmussen, and J. Szefer, “Reading between the dies: Cross-SLR covert channels on multi-tenant cloud FPGAs,” in *37th IEEE ICCD*, 2019, pp. 1–10.
- [11] I. Giechaskiel, K. B. Rasmussen, and J. Szefer, “C3APSULE: Cross-FPGA covert-channel attacks through power supply unit leakage,” in *41st IEEE Symposium on Security and Privacy*, 2020, pp. 1728–1741.
- [12] C. Girling, “Covert channels in LAN’s,” *IEEE Transactions on Software Engineering*, vol. SE-13, no. 2, pp. 292–296, 1987.
- [13] R. Krösche, K. Thimmaraju, L. Schiff, and S. Schmid, “I DPID it my way! a covert timing channel in software-defined networks,” in *17th IFIP Networking*, 2018, pp. 217–225.
- [14] K. S. Lee, H. Wang, and H. Weatherspoon, “PHY covert channels: Can you see the idles?” in *11th USENIX Symposium on Networked Systems Design and Implementation*, Seattle, WA, Apr. 2014, pp. 173–185.
- [15] Department of Computer Science, Cornell University, “SoNIC: Software-defined network interface card,” <http://fireless.cs.cornell.edu/sonic/index.php>, 2023.
- [16] T. Ristenpart, E. Tromer, H. Shacham, and S. Savage, “Hey, you, get off of my cloud: Exploring information leakage in third-party compute clouds,” in *16th ACM CCS*, 2009, p. 199–212.
- [17] P. Pessl, D. Gruss, C. Maurice, M. Schwarz, and S. Mangard, “DRAMA: Exploiting DRAM addressing for Cross-CPU attacks,” in *25th USENIX Security Symposium*, Austin, TX, Aug. 2016, pp. 565–581.
- [18] M. Alagappan, J. Rajendran, M. Doroslovački, and G. Venkataramani, “DFS covert channels on multi-core platforms,” in *2017 IFIP/IEEE International Conference on VLSI-SoC*, 2017, pp. 1–6.
- [19] A. Purnal, F. Turan, and I. Verbauwhede, “Double trouble: Combined heterogeneous attacks on Non-Inclusive cache hierarchies,” in *31st USENIX Security Symposium*, Boston, MA, Aug. 2022, pp. 3647–3664.
- [20] Z. Weissman, T. Tiemann, D. Moghimi, E. Custodio, T. Eisenbarth, and B. Sunar, “Jackhammer: Efficient rowhammer on heterogeneous FPGA-CPU platforms,” *TCHES*, vol. 2020, no. 3, p. 169–195, Jun. 2020.
- [21] Advanced Micro Devices Inc, “Intellectual property,” <https://www.xilinx.com/products/intellectual-property.html>, 2023.
- [22] Intel Corporation, “How to benchmark code execution times on intel® IA-32 and IA-64 instruction set architectures,” <https://www.intel.com/content/dam/www/public/us/en/documents/white-papers/ia-32-ia-64-benchmark-code-execution-paper.pdf>, 2010.
- [23] —, “Intel® 64 and IA-32 architectures software developer’s manual combined volumes: 1, 2a, 2b, 2c, 2d, 3a, 3b, 3c, 3d, and 4,” <https://cdrdv2.intel.com/v1/dl/getContent/671200>, 2023.
- [24] C. Shen, J. Zhang, and G. Qu, “Mes-attacks: Software-controlled covert channels based on mutual exclusion and synchronization,” in *2023 60th ACM/IEEE Design Automation Conference (DAC)*, 2023, pp. 1–6.
- [25] “User Datagram Protocol,” <https://www.rfc-editor.org/info/rfc768>, Aug. 1980.
- [26] H. Huang, X. Wang, Y. Jiang, A. K. Singh, M. Yang, and L. Huang, “On countermeasures against the thermal covert channel attacks targeting many-core systems,” in *2020 57th ACM/IEEE Design Automation Conference (DAC)*, 2020, pp. 1–6.
- [27] J. González-Gómez, M. B. Sikal, H. Khdr, L. Bauer, and J. Henkel, “Smart detection of obfuscated thermal covert channel attacks in many-core processors,” in *2023 60th ACM/IEEE Design Automation Conference (DAC)*, 2023, pp. 1–6.
- [28] K. Sourabh, R. Deepika, V. Lekshmi, S. Sudhakar, and J. Manikandan, “Design and evaluation of BPSK demodulator using model based programming,” in *16th INDICON*, 2019, pp. 1–4.
- [29] J. Zhang, M. Fossorier, D. Gu, and J. Zhang, “Two-dimensional correction for min-sum decoding of irregular LDPC codes,” *IEEE Communications Letters*, vol. 10, no. 3, pp. 180–182, 2006.